How ThreadTrace works

Supporting document · Lindström challenge · Since AI Hackathon 2026 · Felix Liu

In one sentence: ThreadTrace is a live annual review for a Lindström customer. A phone scans a package or garment QR code, an Arduino UNO Q saves the event, and a big screen shows service, repairs, money saved and water saved, with a label on every number.

1. The parts

Phone
Scans a package or garment QR. Adds the place. Reports repairs and write-offs. Takes a photo of a scale.

→

Arduino UNO Q
Small Python server and SQLite on the board. Saves every event first. Calls Gemini to read the scale. Lights the LED matrix.

→

Big screen
The review in a browser. Updates live. Package room, garment story, PDF report.

- **Works offline:** events stay on the board when the network is down.
- **One data file:** a Python script turns the Lindström Excel data pack into `data.json`. To go live, swap this file for approved exports. The screens stay the same.
- **Phone pages:** package page `/p/PK0001`, garment page `/s/A000074`, weigh-in `/weigh`, phone home `/m`. Every phone page has Back, Home and Scan.

2. Package tracking

Scanning every garment is too slow in real work. So each package gets **one big QR code**. The small QR codes on the garments inside belong to the same package.

The package QR knows	The scan adds
Customer, assigned site, style, quantity (max 25), production date, delivery date	The place (customer site or Lindström laundry), the time, and the action: check in, repair or write-off with a number of garments

The demo data has 175 packages, built from the garments in the data pack (same customer, site and style). Production and delivery dates are **demo values**.

3. How we calculate

Number	Formula
Water saved	washed kg × baseline water factor − water Lindström used (allocated). This gives the same values as the data pack's own saving column.

CO_2e saved	washed kg × baseline CO_2e factor − Lindström's process + delivery CO_2e (allocated)
Money saved by repairs	repaired garments × price of a new garment (from the Products sheet). Shown in gold.
CO_2e avoided by repairs	repaired garments × product footprint
Write-off value	written-off garments × price of a new garment (a loss, shown in red)
Weigh-in	kg on the scale × (baseline factor − laundry factor). Lindström's factors are per kg, so nobody counts garments.
On time %	cycles on time ÷ all cycles

Baselines (the customer can choose): outsourced local laundry (default), home washing, in-house laundry.

Ranges: baseline factors ×0.8 / ×1.0 / ×1.2 (low / base / high). Repairs postpone 0.5 / 1 / 1.5 new garments.

4. A label on every number

Label	Meaning	Example
Simulated	Taken from the data pack	98.3% of cycles on time
Estimated	Calculated with a formula you can open	1.41M L water saved (453k–2.36M)
Team assumption	A value we chose, with a reason	±20% range, demo package dates
Public reference	Company-level number from Lindström	About 13% lower emissions intensity than peers
Field scan (demo)	Captured live by the gateway	13.5 kg weighed today

We show numbers we can defend, not the biggest saving.

5. Honest limits and data gaps

- All customer and site data is the **synthetic** Lindström data pack.
- Laundry water and CO_2e are shared out by washed kg. They are not measured per customer.
- **Cleaning and delivery prices are not in the data**, so money saved only counts repairs. We do not guess them.
- Package production and delivery dates are demo values.
- QR codes stand in for the UHF RFID tags Lindström already uses.

- Baseline factors are examples. They need approved values before any claim to a customer.

6. Privacy and security

- No personal data: garments, sites and packages only.
- The Gemini API key is stored on the board in a `.env` file. It is never sent to a browser.
- Events are stored on the board. The network is a private Tailscale network.
- Scale photos are only used to read the weight.

7. From prototype to production

Step	What changes
1. Real data	Replace `data.json` with approved exports (eLindström, Tableau delivery and repair reports, laundry reporting). Same keys, same screens.
2. One pilot site	Package QR codes (or RFID gates) and one gateway at one customer site.
3. Every customer	Many gateways send events (HTTP today, MQTT or an IoT hub later) to one service, for every industry and country.

8. Built with

Arduino UNO Q (Linux + MCU, App Lab, LED matrix) · Python (standard library server, SQLite, pandas for the data build) · Google Gemini API · HTML, CSS, JavaScript, SVG, canvas · Server-Sent Events · Tailscale · qrcodejs and jsQR (open source). AI tools: Claude (Claude Code) for building, Gemini inside the product.

Code: `github.com/FelixLiu339/threadtrace` · README has setup steps.